



Technical Reference Guide

Background Concepts for the IT Utility Toolkit

<Mobile Applications Development>

JP2026 July Online Friendly Competition

Module A(KR) - Phone

This document is supplementary reference material. It explains the concepts and calculation methods behind each tool so that application developers who are not specialists in cryptography or networking can implement the Test Project correctly. It does not contain solution source code.

Contents

Contents	2
1. Introduction	4
2. Foundations: Bytes, Hex and Bitwise Operations.....	4
2.1 Bytes and hexadecimal	4
2.2 Bitwise operations	4
3. Security and Cryptography	5
3.1 Hashing (Hash Generator, File Hash).....	5
Salt	5
File hashing	6
3.2 HMAC (Hash-based Message Authentication Code)	6
3.3 AES Symmetric Encryption	6
Cipher modes: how blocks are chained.....	6
The Initialization Vector (IV)	6
Padding (PKCS7).....	7
Putting it together	7
4. Encoding and Conversion	7
4.1 Character encoding: ASCII and UTF-8.....	7
4.2 Base64	7
4.3 URL Encoding (Percent-encoding, RFC 3986)	8
4.4 Number Base Converter	8
4.5 ASCII / HEX Converter	8
5. Networking	9
5.1 IPv4 addresses	9
5.2 Subnet masks and CIDR notation	9
5.3 CIDR calculation, step by step	9
Worked example 1 — 192.168.1.10 /24	9
Worked example 2 — 172.16.5.130 /27 (subnetting inside an octet)	10
Subnet mask quick reference	10
5.4 IP Address Calculator	10
5.5 Ports (Port Checker)	10
6. General Tools.....	11
6.1 UUID Generator	11
6.2 Password Generator	11
6.3 Regular Expression Tester	11
6.4 Random Number Generator.....	12

Appendix A. Quick Reference Tables..... 13

 A.1 Powers of two 13

 A.2 Hex ↔ binary nibbles..... 13

 A.3 Selected printable ASCII codes..... 13

 A.4 Values used in the sample test vectors..... 13

1. Introduction

The IT Utility Toolkit gathers fifteen small utilities that IT developers and security engineers use every day. Each tool looks simple on screen, but several of them rely on standard algorithms from cryptography, character encoding and IP networking. This guide describes what those algorithms do, why they exist and how the results are calculated, with worked examples that match the values in the provided test-vectors.json file.

How to read this guide:

- Each section first explains the concept in plain language.
- “How it is calculated” subsections give the exact method, step by step.
- Worked examples use the same inputs shown in the application prototypes.
- Reference tables at the end summarise the most-used values (powers of two, subnet masks, common ports, ASCII codes).

A note on representation. Throughout the toolkit, three ideas appear again and again:

- Everything is bytes. Text, files, keys and IP addresses are ultimately sequences of 8-bit bytes (values 0–255).
- Text becomes bytes through an encoding. This toolkit always uses UTF-8.
- Bytes are shown to humans as hexadecimal (base 16) or Base64, because raw bytes are not printable.

2. Foundations: Bytes, Hex and Bitwise Operations

These basics are used by almost every tool, and they are essential for the networking calculations later in the guide.

2.1 Bytes and hexadecimal

A byte holds a value from 0 to 255. Written in binary that is eight bits, for example 1100 0000. Because long binary strings are hard to read, one byte is usually written as two hexadecimal digits, where each digit represents four bits (a “nibble”).

Binary (4 bits)	Hex	Decimal
0000	0	0
0101	5	5
1010	A	10
1100	C	12
1111	F	15

Example: the byte 1100 0000 = hex C0 = decimal 192.

2.2 Bitwise operations

Bitwise operators work on the individual bits of a value. Three of them are the heart of the CIDR calculation.

Operator	Rule	Example (per bit)
AND (&)	result bit is 1 only if BOTH bits are 1	1100 & 1010 = 1000
OR ()	result bit is 1 if EITHER bit is 1	1100 1010 = 1110
NOT (~)	inverts every bit (0↔1)	~1100 = 0011
Shift left (<<)	moves bits left, filling with 0	0001 << 3 = 1000

AND is used to keep some bits and zero the rest (“masking”). OR is used to force bits to 1. These are exactly the steps used to find a network address and a broadcast address.

3. Security and Cryptography

3.1 Hashing (Hash Generator, File Hash)

A hash function takes any input — a word, a paragraph, or an entire file — and produces a fixed-size “fingerprint” called a digest. The same input always gives the same digest, but you cannot work backwards from the digest to the input.

Key properties:

- Deterministic: identical input always produces identical output.
- Fixed size: the output length depends only on the algorithm, not on the input size.
- One-way: it is computationally infeasible to reverse the digest back to the input.
- Avalanche effect: changing a single character changes roughly half of the output bits.
- Collision resistance: it should be extremely hard to find two different inputs with the same digest.

Typical uses are file integrity checks (has this download been altered?), storing password verifiers, and de-duplication.

Algorithms and their output sizes:

Algorithm	Digest size	Hex characters	Notes
MD5	128 bits (16 bytes)	32	Fast; broken for collisions — use only for checksums
SHA-1	160 bits (20 bytes)	40	Deprecated for security use
SHA-224	224 bits (28 bytes)	56	SHA-2 family
SHA-256	256 bits (32 bytes)	64	SHA-2 family; very common today
SHA-384	384 bits (48 bytes)	96	SHA-2 family
SHA-512	512 bits (64 bytes)	128	SHA-2 family

The digest is a sequence of bytes; the toolkit displays it as lowercase hexadecimal. For example the 16 bytes of an MD5 digest become 32 hex characters.

Worked example — MD5 of WorldSkills:

```
input   : WorldSkills   (UTF-8 bytes: 57 6F 72 6C 64 53 6B 69 6C 6C 73)
MD5      : e514b0a3298d6efe1322b85ddb91a791
SHA-256: 7a474640b9e0fc9aa42fd27d92f5ffe331b25b0ab5e0537d2bd6d7ee387ffd75
```

Salt

A salt is a short random value combined with the input before hashing. Because the salt differs every time, two identical inputs produce different digests, which defeats precomputed “rainbow table” attacks. In this toolkit the salt is a random 16-byte value shown as 32 hexadecimal characters, and it is joined to the input as text before hashing:

```
hashInput = UTF8( inputText + displayedSaltText )
digest    = SHA-256( hashInput )
```

File hashing

Hashing a file works exactly like hashing text, except the input bytes come from the file content rather than from typed characters. Comparing a freshly computed hash with a published one proves the file was downloaded without corruption or tampering. The file size shown in the app is simply bytes ÷ 1024, rounded to two decimals (for example 49 bytes = 0.05 KB).

3.2 HMAC (Hash-based Message Authentication Code)

A plain hash proves that data has not changed, but anyone can compute it, so it cannot prove WHO produced the data. HMAC solves this by mixing a secret key into the hashing process. Only someone who knows the key can generate — or verify — the correct code. HMAC therefore provides both integrity and authenticity.

Conceptually the message and a key are combined and hashed twice:

```
HMAC(key, message) =  
  H( (key XOR outerPad) + H( (key XOR innerPad) + message ) )
```

“H” is any hash function, which is why the tool offers HMAC-MD5, HMAC-SHA1, HMAC-SHA256 and HMAC-SHA512. Real-world uses include signing API requests, validating webhooks, and the signature part of JSON Web Tokens (JWT). Input and key are treated as UTF-8, and the result is shown as lowercase hexadecimal.

3.3 AES Symmetric Encryption

AES (Advanced Encryption Standard) is the most widely used symmetric cipher. “Symmetric” means the same secret key is used to encrypt and to decrypt. Unlike a hash, encryption is reversible: with the correct key the original text is recovered exactly.

AES processes data in fixed 16-byte (128-bit) blocks. The key may be 128, 192 or 256 bits long (16, 24 or 32 bytes); a longer key means more internal rounds and a larger key space.

Algorithm	Key length	Block size
AES-128	16 bytes	16 bytes
AES-192	24 bytes	16 bytes
AES-256	32 bytes	16 bytes

Cipher modes: how blocks are chained

A block cipher only knows how to encrypt one 16-byte block. A “mode of operation” defines how a longer message is split into blocks and combined. The toolkit offers two classic modes.

- ECB (Electronic Codebook): every block is encrypted independently. Simple, but identical plaintext blocks produce identical ciphertext blocks, which leaks patterns. It does not use an IV and is not recommended for real data — it is included here for learning and comparison.
- CBC (Cipher Block Chaining): before each block is encrypted it is XOR-combined with the previous ciphertext block. The very first block has no previous block, so it is XOR-combined with an Initialization Vector (IV) instead. This makes identical plaintext produce different ciphertext, hiding patterns.

```
CBC encryption:  
C0 = Encrypt( P0 XOR IV )  
C1 = Encrypt( P1 XOR C0 )  
C2 = Encrypt( P2 XOR C1 )    ...and so on
```

The Initialization Vector (IV)

The IV is a 16-byte value (the same size as one block). It does not need to be secret and is normally stored or sent together with the ciphertext, but for CBC it should be unpredictable and different for each encryption. In this toolkit the IV is typed as 32 hexadecimal characters (16 bytes × 2 hex digits). The IV field is shown only for

CBC mode; ECB does not use one. To decrypt a CBC message you must supply the same IV that was used to encrypt it.

Padding (PKCS7)

Because AES works on whole 16-byte blocks, the plaintext must be padded to a multiple of 16 bytes. PKCS7 padding appends N bytes, each having the value N, where N is the number of padding bytes needed (1–16). If the data is already a multiple of 16, a full extra block of value 16 (0x10) is added, so that decryption can always remove the padding unambiguously.

Putting it together

The encryption pipeline used by the tool is:

1. Convert the plain text to UTF-8 bytes.
2. Pad the bytes to a multiple of 16 using PKCS7.
3. Encrypt with the chosen key size and mode (CBC uses the IV; ECB does not).
4. Encode the resulting cipher bytes as standard Base64 (no line breaks) for display.

Decryption reverses the process: Base64-decode the input, decrypt the blocks, remove the PKCS7 padding, and read the bytes back as UTF-8 text.

Worked example — AES-128 / CBC:

```
plain text : Hello WorldSkills
key (ASCII): 1234567890ABCDEF          (16 bytes → AES-128)
IV (hex)   : 000102030405060708090A0B0C0D0E0F
ciphertext : n2tQ3xoA0rRAUoInZAeHLBjD4S6g0VhQwrHfkXcI/Hw= (Base64)
```

If you re-run the same inputs you will get exactly this Base64 string. Change the IV and the ciphertext changes completely, even though the key and text are the same.

4. Encoding and Conversion

Encoding is not encryption. It changes the representation of data so it can travel safely through a text-only channel; anyone can reverse it. No key is involved.

4.1 Character encoding: ASCII and UTF-8

A computer stores text as numbers called code points. ASCII defines the first 128 code points (0–127): the English letters, digits and common punctuation. UTF-8 is a superset that can represent every character in every language by using one to four bytes per character; the first 128 code points are identical to ASCII (one byte each). This toolkit uses UTF-8 everywhere, which is why the ASCII/HEX tool restricts input to the ASCII range while hashing and encryption accept any UTF-8 text.

4.2 Base64

Base64 turns arbitrary bytes into printable text using 64 safe characters (A–Z, a–z, 0–9, “+” and “/”). It is used to embed binary data — an image, a key, a ciphertext — inside text formats such as JSON, e-mail or a URL.

How it is calculated: the input bytes are taken three at a time. Three bytes are 24 bits; those 24 bits are re-grouped into four 6-bit numbers, and each 6-bit number (0–63) selects one character from the alphabet. When the input length is not a multiple of three, one or two “=” padding characters are added at the end.

Leftover input bytes	Output	Padding
3 bytes	4 characters	none
2 bytes	3 characters + “=”	one “=”
1 byte	2 characters + “==”	two “=”

Worked example: WorldSkills 2026 → V29ybGRTa2lsbHMgMjAyNg==

The toolkit uses the standard alphabet (not the URL-safe “-” variant) and never inserts line breaks. Decoding reverses the mapping; input that is not valid Base64, or that does not decode to valid UTF-8 text, is rejected with an error.

4.3 URL Encoding (Percent-encoding, RFC 3986)

URLs may only contain a limited set of characters. Percent-encoding replaces every unsafe byte with a “%” followed by its two-digit hexadecimal value. The rules followed here come from RFC 3986:

- Unreserved characters are left unchanged: A–Z, a–z, 0–9, hyphen “-”, period “.”, underscore “_”, and tilde “~”.
- Every other byte is percent-encoded using its UTF-8 value, with uppercase hex digits (e.g. “/” → %2F).
- A space becomes %20. (The “+” sign means space only in HTML form submissions, a different rule — it is not used here.)

Worked example:

```
input : https://worldskills.org/?q=mobile app&lang=ko
output: https%3A%2F%2Fworldskills.org%2F%3Fq%3Dmobile%20app%26lang%3Dko
```

4.4 Number Base Converter

Numbers can be written in different bases. In any base *b*, each digit position is worth a power of *b*. The toolkit converts between four bases:

Base	Name	Digits used	Bits per digit
2	Binary	0–1	1
8	Octal	0–7	3
10	Decimal	0–9	—
16	Hexadecimal	0–9 and A–F	4

How it is calculated. To go to decimal, multiply each digit by its place value and add them up. To go from decimal to another base, divide repeatedly by the base and read the remainders bottom-to-top. Hexadecimal maps neatly onto binary because one hex digit equals exactly four bits, and one octal digit equals three bits.

Worked example — the binary number 1111111:

```
1111111 (binary) = 1·64 + 1·32 + 1·16 + 1·8 + 1·4 + 1·2 + 1·1 = 127
decimal       : 127
octal        : 177      (127 = 2·64 + ... → 1 7 7)
hexadecimal  : 7F       (127 = 7·16 + 15)
```

This toolkit treats numbers as unsigned 32-bit integers, so the valid range is 0 to 4,294,967,295. Hexadecimal output is shown in uppercase.

4.5 ASCII / HEX Converter

This tool shows the numeric code of each character. In “Text → Hex” mode each character is replaced by its two-digit hexadecimal code; in “Hex → Text” mode the reverse happens. Because it works one byte per character, it accepts only ASCII characters (codes 00–7F).

Worked example: Hello → 48 65 6C 6C 6F (H=0x48, e=0x65, l=0x6C, l=0x6C, o=0x6F)

5. Networking

This is the section most application developers find unfamiliar, so it is covered in the most detail. Everything here is IPv4 — the classic four-number addresses.

5.1 IPv4 addresses

An IPv4 address is a 32-bit number, normally written as four decimal “octets” (bytes) separated by dots, each from 0 to 255. For example 192.168.1.10. The four octets are simply the four bytes of one 32-bit value.

Converting an address to a single 32-bit integer is the key trick that makes all the calculations easy — you do arithmetic on the integer, then convert back.

```
192.168.1.10
= 192·2563 + 168·2562 + 1·256 + 10
= 192·16777216 + 168·65536 + 1·256 + 10
= 3,232,235,786

In binary (32 bits, one octet per group):
11000000 . 10101000 . 00000001 . 00001010
```

To convert back, split the 32-bit value into four bytes (shift right by 24, 16, 8 and 0 bits, keeping the low 8 bits each time).

5.2 Subnet masks and CIDR notation

A network is a block of consecutive addresses. Every address is split into a network part (identifies the block) and a host part (identifies a device inside the block). CIDR notation writes the size of the network part as a slash and a number, for example /24, meaning “the first 24 bits are the network part”.

The subnet mask expresses the same idea as a 32-bit value: it is all 1-bits for the network part, followed by all 0-bits for the host part.

```
/24 → 11111111 11111111 11111111 00000000 = 255.255.255.0
/26 → 11111111 11111111 11111111 11000000 = 255.255.255.192
```

How the mask is built: take a value of all ones (32 bits) and shift it left by (32 – prefix) positions, so that exactly “prefix” one-bits remain on the left.

5.3 CIDR calculation, step by step

Given an IP address and a prefix length /n, the calculator produces six results. Here is the complete method.

5. Convert the IP address to its 32-bit form (see 5.1).
6. Build the subnet mask: n one-bits followed by (32 – n) zero-bits.
7. Network address = IP AND mask. The AND clears every host bit to 0, leaving the block’s starting address.
8. Broadcast address = Network OR (NOT mask). NOT mask is 0 for the network part and 1 for the host part, so OR sets every host bit to 1 — the block’s last address.
9. First host = Network address + 1 (the network address itself is not assigned to a device).
10. Last host = Broadcast address – 1 (the broadcast address is reserved).
11. Number of usable hosts = 2^(32 – n) – 2. The total number of addresses in the block is 2^(32 – n); subtract 2 for the network and broadcast addresses.

Worked example 1 — 192.168.1.10 /24

```
IP       : 11000000 10101000 00000001 00001010   (192.168.1.10)
Mask /24 : 11111111 11111111 11111111 00000000   (255.255.255.0)

Network  = IP AND mask
          : 11000000 10101000 00000001 00000000   → 192.168.1.0
Broadcast = Network OR (NOT mask)
```

```

      : 11000000 10101000 00000001 11111111 → 192.168.1.255
First host: 192.168.1.1      Last host: 192.168.1.254
Usable hosts = 2^(32-24) - 2 = 2^8 - 2 = 254

```

Worked example 2 — 172.16.5.130 /27 (subnetting inside an octet)

When the prefix is not a multiple of 8, the split happens inside one octet. A /27 mask ends with 11100000 in the last octet = 224, so the last octet mask is 255.255.255.224. The “block size” is $256 - 224 = 32$, which means each subnet is 32 addresses wide: .0, .32, .64, .96, .128, .160, .192, .224. The address .130 falls in the .128–.159 block.

```

IP       : ... 10000010 (last octet 130)
Mask /27 : ... 11100000 (255.255.255.224, block size 32)

Network  = 172.16.5.128 (130 AND 224 = 128)
Broadcast = 172.16.5.159 (128 + 32 - 1)
First host: 172.16.5.129 Last host: 172.16.5.158
Usable hosts = 2^(32-27) - 2 = 2^5 - 2 = 30

```

Shortcut for the last octet: block size = $256 - (\text{last-octet mask value})$. The network’s last octet is the largest multiple of the block size that is $\leq$ the address’s last octet.

Subnet mask quick reference

CIDR	Subnet mask	Block size	Usable hosts
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

5.4 IP Address Calculator

Once an address is a 32-bit integer, arithmetic is straightforward, and carrying between octets happens automatically because it is really one big number.

- Add / Subtract: convert to the integer, add or subtract the value, convert back. Adding must not exceed 255.255.255.255 (overflow) and subtracting must not go below 0.0.0.0 (underflow).
- Count: the number of addresses from a start IP to an end IP, inclusive, is $(\text{end} - \text{start}) + 1$.

```

Add       : 192.168.1.250 + 10
           250 + 10 = 260 → carry into the third octet
           result: 192.168.2.4
Count     : 192.168.1.1 to 192.168.1.10
           (10 - 1) + 1 = 10 IP addresses

```

5.5 Ports (Port Checker)

A single computer runs many network services at once. A port number (0–65535) identifies which service a connection is meant for. Ports 0–1023 are the “well-known” ports reserved for common services. The Port

Checker screen in this toolkit is a wireframe only: a browser or sandboxed app cannot open raw TCP connections, so no real check is performed.

Service	Port	Service	Port
HTTP	80	SSH	22
HTTPS	443	FTP	21
DNS	53	SMTP	25

6. General Tools

6.1 UUID Generator

A UUID (Universally Unique Identifier) is a 128-bit number used as an identifier that can be generated independently on many machines with practically no chance of collision. It is written as 32 hexadecimal digits in five groups separated by hyphens: 8-4-4-4-12.

```
xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx
           ^   ^
           |   | variant nibble: one of 8, 9, a, b
           |   | version nibble: 4 (random-based UUID)
```

Version 4 UUIDs are filled with random bits, except two fixed markers: the version digit is always “4”, and the first digit of the fourth group is 8, 9, a or b. The output is lowercase, and two consecutive generations are effectively never equal.

6.2 Password Generator

Password strength is measured as entropy, in bits. Each additional character multiplies the number of possibilities by the size of the character pool, so entropy grows with both the length and the variety of characters:

```
entropy (bits) ≈ length × log2( pool size )

pool sizes:  lowercase 26,  +uppercase 52,  +digits 62,
              +symbols ≈ 94
```

The generator lets the user choose the length (4–64) and which character sets to include. To honour every choice, the result is guaranteed to contain at least one character from each selected set, and only characters from the selected sets.

6.3 Regular Expression Tester

A regular expression (regex) is a compact pattern language for describing text. The tester reports whether a test string matches a pattern. This toolkit performs a partial search: the result is “Matched” when the pattern is found anywhere inside the test string. To require the whole string to match, anchor the pattern with “^” at the start and “\$” at the end.

Token	Meaning	Token	Meaning
^ ... \$	start ... end of string	\d	a digit 0–9
.	any one character	\w	a word character
*	zero or more	+	one or more
{m,n}	between m and n times	[abc]	any listed character

Example: `^[a-zA-Z0-9_]{6,16}$` matches a username of 6–16 letters, digits or underscores.

6.4 Random Number Generator

The generator produces whole numbers spread uniformly across a range. A value in the range [min, max] is obtained by taking a raw random number and mapping it: $\text{min} + \text{random} \cdot (\text{max} - \text{min} + 1)$. Each value is independent, so duplicates can occur, and both negative and positive bounds are allowed. For security-sensitive uses (keys, salts, passwords) a cryptographically secure random source should be used rather than an ordinary one.

Appendix A. Quick Reference Tables

A.1 Powers of two

n	2^n	n	2^n
0	1	10	1,024
1	2	16	65,536
4	16	24	16,777,216
8	256	32	4,294,967,296

A.2 Hex ↔ binary nibbles

Hex	Binary	Dec	Hex	Binary	Dec
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

A.3 Selected printable ASCII codes

Char	Hex	Dec	Char	Hex	Dec
(space)	20	32	A	41	65
0	30	48	H	48	72
9	39	57	a	61	97
@	40	64	o	6F	111

A.4 Values used in the sample test vectors

These inputs and outputs appear in the application prototypes and in test-vectors.json. They can be used to check an implementation.

Tool	Input	Expected output
MD5	WorldSkills	e514b0a3298d6efe1322b85ddb91a791
SHA-256	WorldSkills	7a474640...387ffd75 (64 hex)
HMAC-SHA256	text WorldSkills, key secret	dd9e9b2e...22ce6ee8 (64 hex)
AES-128 CBC	Hello WorldSkills	n2tQ3xoA0rRAUoInZAeHLBjD4S6g0VhQwrHfkXcl/Hw=
Base64	WorldSkills 2026	V29ybGRTa2lsbHMgMjAyNg==
CIDR /24	192.168.1.10/24	net .0, bcast .255, 254 hosts
IP add	192.168.1.250 + 10	192.168.2.4

— *End of Technical Reference Guide* —